

Laboratorio di Architetture Software e Sicurezza Informatica [AAF1569]

4 - SaaS essentials



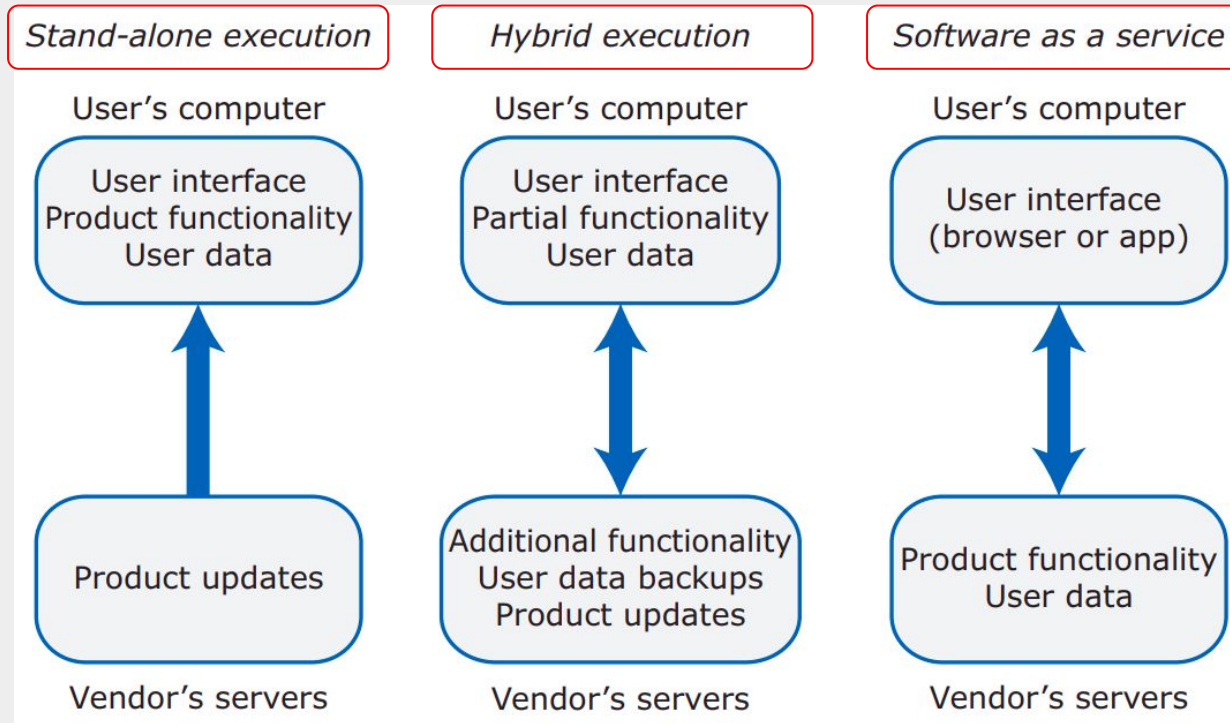
SAPIENZA
UNIVERSITÀ DI ROMA

Part of therein contents are based on slides of Prof, John Yang and Ian Sommerville

DIAG

Dipartimento di Ingegneria
informatica, automatica e gestionale
Antonio Ruberti

Software Execution Models



SaaS Advantages

1. **No local installation:** removes hardware compatibility requirements
2. **Data is kept within a service** (no backup/data loss concerns for clients)
3. A **single copy** of server software runs in an environment chosen and controlled by developers (e.g., no need to worry about specific OS)
4. **Upgrades** can be handled so as not to interfere with user experience
5. Easier for groups of users to **collectively interact** on same data
6. Remote access is handy for **large or frequently updated data**

Disadvantages?

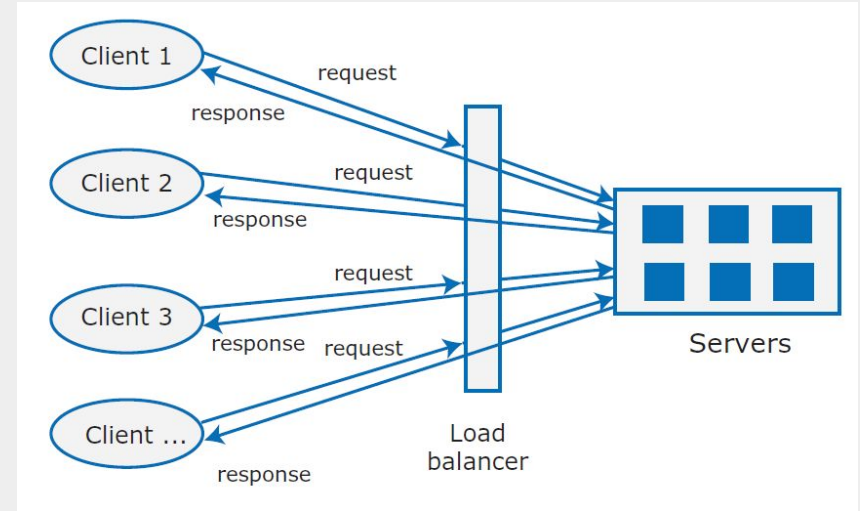
Client-Server

Clients are programs whose specialty is asking servers for information and (usually) allowing the user to interact with that information

Servers are programs whose specialty is efficiently serving large numbers of clients simultaneously.

A server **appears as a single logical entity** to the client, which can remain blissfully unaware of the server's deployment topology.

Distinguishing clients from servers allows each type of program to be highly **specialized to its task**: the client can have a responsive and appealing user interface, while the server concentrates on efficiently serving many clients simultaneously



How do SaaS clients and SaaS servers communicate?

The Internet **protocol suite**, commonly known as **TCP/IP**, is the set of communications protocols used in the Internet and similar computer networks.

- IP (DNS)
- TCP (TLS)
- HTTP(S)

HTTP

The HTTP protocol: **rules for communication** to make a request and receive a response

1. The client **initiates a TCP/IP connection** to a server by specifying the IP address and port number (usually 80).
2. if the connection succeeds, the client immediately sends an **HTTP request** describing its intention to perform some operation on a resource.
3. The server delivers an **HTTP response** either satisfying the client's request or reporting any errors that prevented the request from succeeding. The response may also include information in the form of an **HTTP cookie** that allows the server to correctly identify this same client on future interactions.

HTTP

ROUTE

REQUEST

[HTTP verb]	[URI]	[HTTP version]
[Request header]		
[Request body]		

RESPONSE

[HTTP version]	[Response code]
[Response header]	
[Response body]	

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

HTTP

REQUEST

GET	incidents/A90/stonehaven/	HTTP/1.1
Host: trafficinfo.net		
...		
Accept: text/json, text/xml, text/plain		
Content-Length: 0		

RESPONSE

HTTP/1.1	200
...	
Content-Length: 461	
Content-Type: text/json	
{	
"number": "A90N17061714391",	
"date": "20170617",	
"time": "1437",	
"road_id": "A90",	
"place": "Stonehaven",	
"direction": "north",	
"severity": "significant",	
"description": "Broken-down bus on north	
carriageway. One lane closed. Expect	
delays of up to 30 minutes."	
}	
{	
"number": "A90S17061713001",	
"date": "20170617",	
"time": "1300",	
"road_id": "A90",	
"place": "Stonehaven",	
"direction": "south",	
"severity": "minor",	
"description": "Grass cutting on verge.	
Minor delays"	
}	

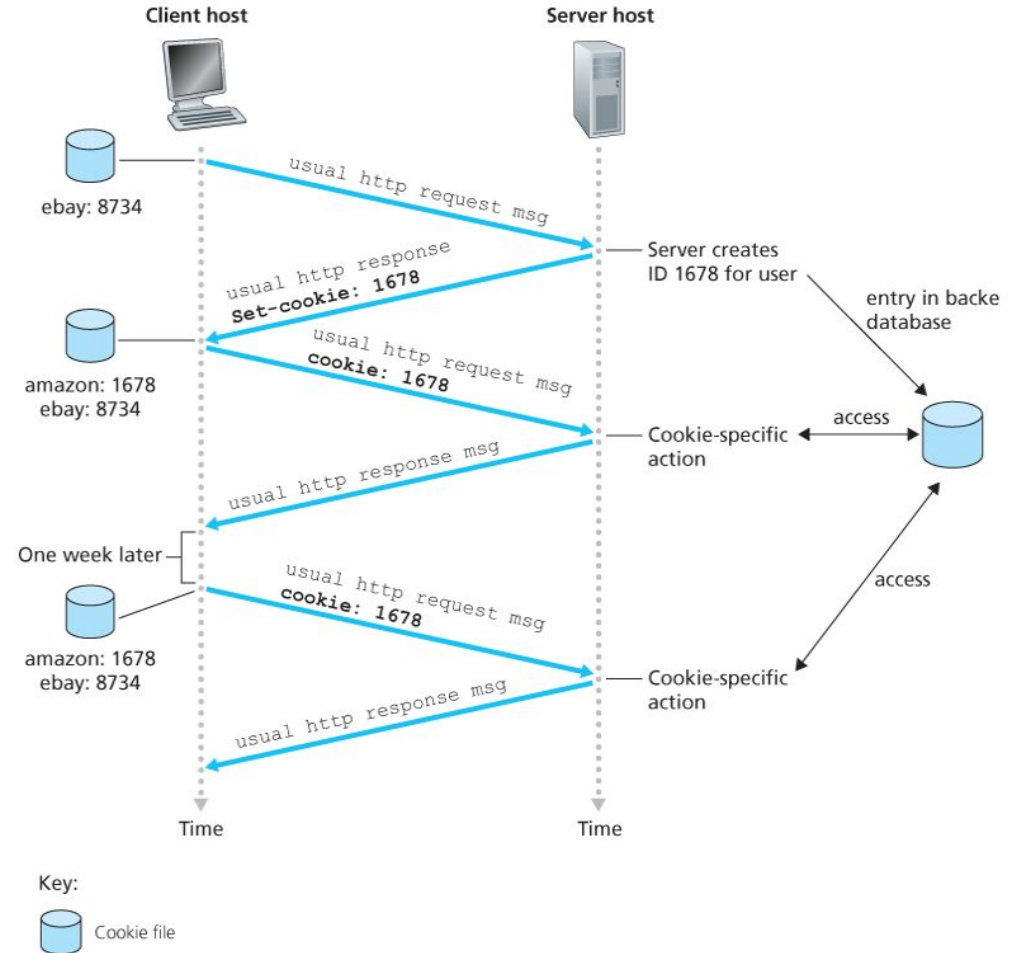
HTTP

HTTP is a **stateless protocol** : every HTTP request is independent of and unrelated to all previous requests.

State _> Cookie The first time a client makes a request to a particular server, the server can include in the response a **Set-Cookie:** header, containing a chunk of information that the server can use to identify this client on future HTTP requests

It is the client's responsibility to store that information and pass it back via a **Cookie:** header on every subsequent request to that same server.

HTTP interactions and Cookies



Shift of Web Technologies

When the Web began in 1990, **HTTP servers** existed primarily to serve **static content** that browsers would display. Each time the browser made another HTTP request, the server would deliver a new web page for the browser to show.

But the **emergence of SaaS around 1995** soon shifted the functionality of servers: rather than simply returning copies of static Web pages, **servers would now run a program, and create HTML pages on the fly** that implemented that program's user interface.

Shift of Web Technologies

The next evolutionary step was the appearance of **AJAX**, or **Asynchronous JavaScript And XML**. If a browser supported AJAX, pages could include code written in the JavaScript language, and that code could make **subsequent HTTP requests** to the server **without causing a page reload**.

In response to those requests, the server would return not an HTML page, but a **data structure in XML or JSON format**. The JavaScript code running in the browser would use that data to determine **how to change the appearance or behavior of the displayed page**, all without causing a page reload.

Shift of Web Technologies

AJAX marked a turning point in the relationship between a Web site and a client: instead of receiving HTML pages and functioning primarily as a display engine, the client was calling a function on the remote server and expecting to get some data back, *just as if the client was calling a library function*.

This shift in perspective invited a **new way of looking at the Web**: as a set of independent services that could be composed to produce larger sites.

A so-called **Service Oriented Architecture (SOA)**.

Service Oriented Architecture needs

Service Oriented Architecture (SOA): an approach to software development in which subsystems can only access each others' data via external interfaces.

Each service provides a well-defined set of operations on one or a few related types of resources

SaaS apps as “services” that can accept remote procedure calls from clients

_> We **need** to be able to **identify the service**, identify which **operation** is to be called, **pass data** to and **receive data** from the service, and **handle errors**.

API definition

The term API, or Application Programming Interface, refers to the “**contract**” **between a caller and callee**, whether these are a program calling a library function or a SaaS client invoking a service on a SaaS server.

API documentation specifies which operations are available, **how URIs should be constructed** to invoke those operations, **how errors are reported**, and what other **operational requirements** must be met (for example, limiting the number of calls made per day to a microservice, or including an account identifier or password with each API call)

REST: REpresentational State Transfer

REST is not a standard, software architectural style: it is a **design stance** on how a service should be constructed and, by extension, what its API should look like

- Represent the various entities manipulated by a Web app as **resources**
- Construct routes so that any **HTTP request would contain all the information necessary to identify both a resource and the action to be performed** on it (**stateless**)

An API guidelines is said to be RESTful, and the **routes** (HTTP method + URI) defined by the API to invoke particular actions are said to be RESTful routes.

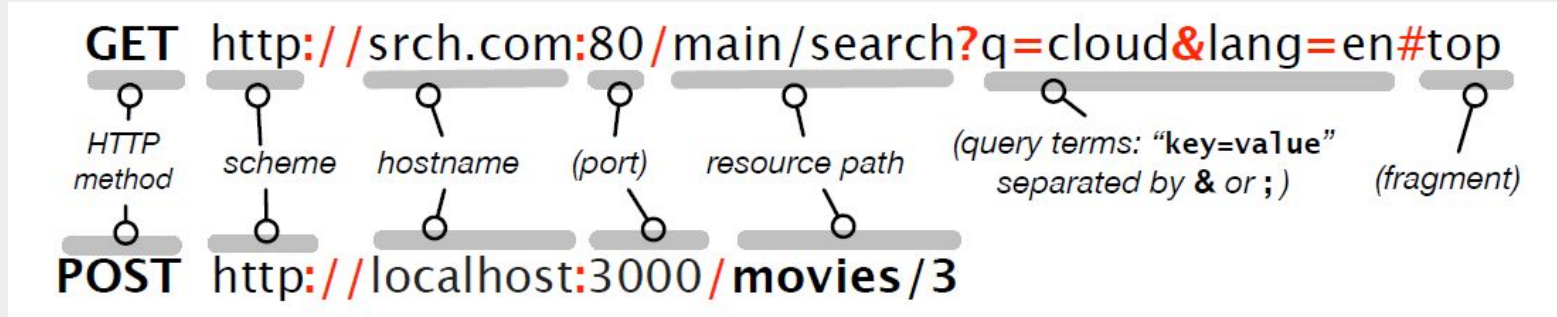
RestFul API - CRUDI

In general, **RESTful API** defines up to five operations on a resource, captured by the acronym **CRUDI**:

- **C**reate a new instance of a resource,
- **R**etrieve a copy of a resource,
- **U**ppdate a resource,
- **D**eleate a resource,
- list an **I**ndex of all available resources of a given type, possibly filtered by particular criteria.

Nearly always, each resource is given a **unique ID** (usually a number) that will serve as the “permanent handle” to that resource and is never reused even if the resource is deleted.

HTTP Route

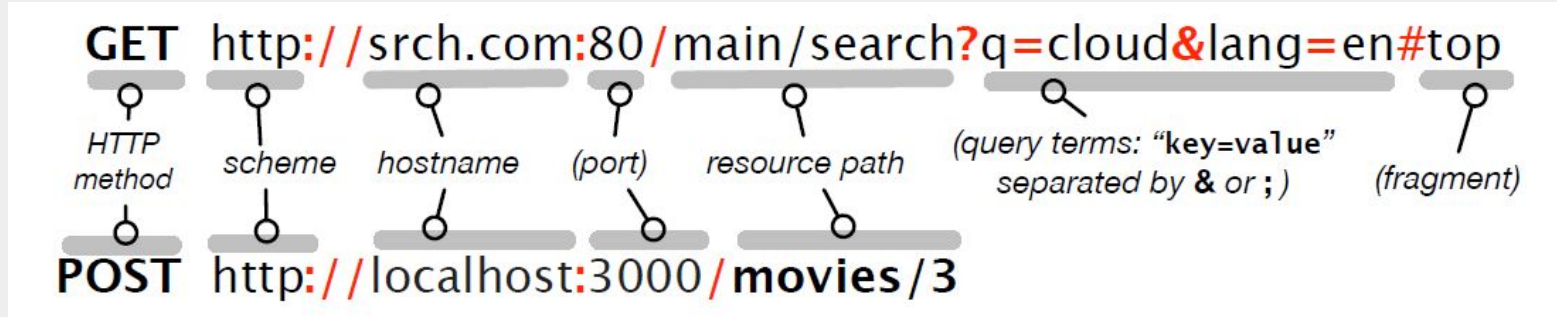


HTTP method - usually, one of *GET*, *POST*, *PUT*, *PATCH*, or *DELETE*

URI - Uniform Resource Identifier, names a resource available on the Internet

The same URI with different HTTP methods can have different meanings to a SaaS app

HTTP Route



GET typically means “deliver a copy of the resource to the client without modifying the resource or causing any side effects”.

POST, PUT, PATCH, DELETE are typically used to perform an operation that creates, modifies, or deletes a resource.

RESTful URI

RESTful APIs observe several conventions for calls during URI construction

- The hostname component of the URI tells us which server provides the service: for example, <https://api.themoviedb.org> (from the book)
- Most servers specify a **base URI** (a common URI prefix) that should be prepended to all API calls: e.g., <https://api.themoviedb.org/4/> - this is also called the **API endpoint** that clients contact in order to use the service
- The URI path **components following the prefix** specify the **operation** to be performed and the **resource** on which to perform it

RESTful URI

API documentation frequently uses a **colon :** or **curly braces {}** to indicate a URI component corresponding to a resource ID,

The API documentation might state that the route `GET /movie/{movie_id}` or `GET /movie/:movie_id` requests detailed information for the specific movie whose numeric ID is substituted for `{movie_id}` or `:movie_id` in the URI.

<https://developer.spotify.com/documentation/web-api/reference/#/operations/get-an-artists-albums>

RESTful URI

	Non-RESTful site URI	RESTful site URI
1. Login to site	POST /login/dave	POST /login/dave
2. Welcome page	GET /welcome	GET /user/301/welcome
3. Add item ID 427 to cart	POST /add/427	POST /user/301/add/427
4. View cart	GET /cart	GET /user/301/cart
5. Checkout	POST /checkout	POST /user/301/checkout

In the **non-RESTful site** on the left, every operation after the login **relies on implicit information** (login on step 2, cart con step 5 - both uniquely identified by user ID 301 in the RESTful site on the right)

How parameters and data are exchanged?

Three ways to pass parameters from an HTTP client to a service:

1. in the **URI** (GET /search/movies?query=Batman+Returns)
2. in the **request body** (for POST or PUT requests)
3. (rarely) as the value of an **HTTP header**

For few parameters, they may be embedded in the URI.

When the data to pass is more complex, or when the API operation involves a state-changing HTTP method such as POST or PUT, the data is sent as part of the request body (recall: body of GET requests is commonly ignored)

How is this data presented to the server? **JSON** (JavaScript Object Notation) is the common interchange format for the SOA community

JSON (JavaScript Object Notation)

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [],
  "spouse": null
}
```

Standard **data interchange** (file) format

Essentially: a set of unordered key/value pairs

Key (or “slot”): double-quoted string

Value: a simple type (string, numeric, `true`, `false`, `null`), a linear array (each of whose element can be any of these), or another object (whose slots are constrained to these same types)

Whitespace (spaces, tabs, newlines) is optional. Most servers return whitespace-free JSON.

API authorization and requirements

Nearly all APIs require **authorization**: the client must prove it has the right to make each API call.

While authorization schemes vary, the most common is to include a client-specific **API key** with each request. It can be supplied in the URI, as the value of an HTTP `Authorization:` header, or both.

API keys are usually requested manually and may be free or paid.

A service may impose limits, such as the number of calls made per day.

HTTP response status codes

Services use **status codes** to indicate various **types of errors**:

- **2xx** codes: **success**
- **3xx** codes: client should **take further action** (e.g., redirect to complete)
- **4xx** codes: service encountered an application **error processing the request** (e.g., 400 Bad Request, 401 Unauthorized, 402 Payment Required)
- **5xx** codes: **problem with the service infrastructure** itself—an error that prevented the remote call from even complet

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

In case of an error, the response body usually contains a message explaining what went wrong.

RESTful design

Think in term of resources. RESTful API design approach should begin by asking:

- What **kind** of resource?
- What **operations** are available on it?
- **What can go wrong** during each type of operation?
- How is each kind of **resource stored** on the server, and under what **conditions** can it be **deleted**?
- How does the client **refer** to a particular **resource** that was **created earlier**, even if the user has been away from the keyboard for a long time?

A common pitfall is instead of thinking in terms of URIs, user actions, and views (as in tracking what the user is doing and providing logic to keep up with it...)

Model View Controller (MVC)

Design pattern for the architecture/organization of the server application

An application organized according to MVC consists of three main types of code:

1) **Models** (core functionality and data)

- They are concerned with the data manipulated by the application: how to store it, how to operate on it, and how to change it
- Typically, **one model for each type of entity** manipulated by the MVC app

Model View Controller (MVC)

Design pattern for the architecture/organization of the server application

An application organized according to MVC consists of three main types of code:

2) **Views** (display information to the user)

- They are presented to the user and contain information about the models with which users can interact.
- The views serve as the **interface between** the system's **users and** its **data**;

Model View Controller (MVC)

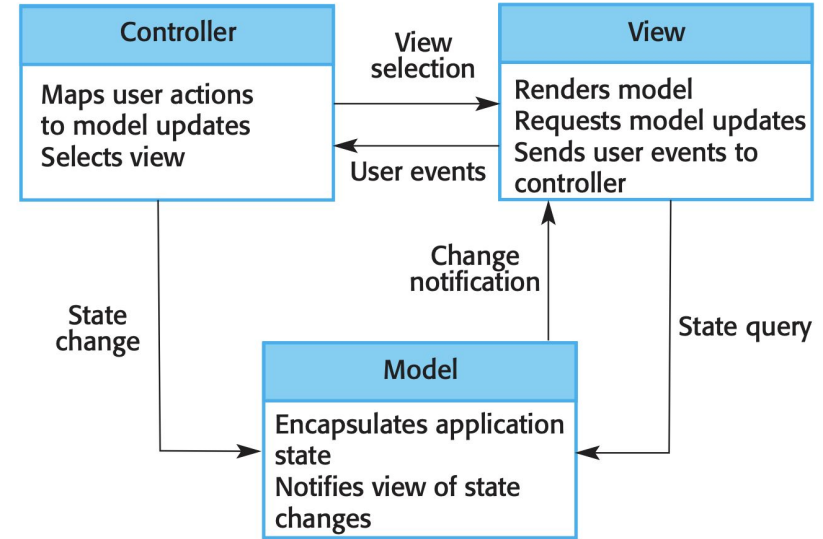
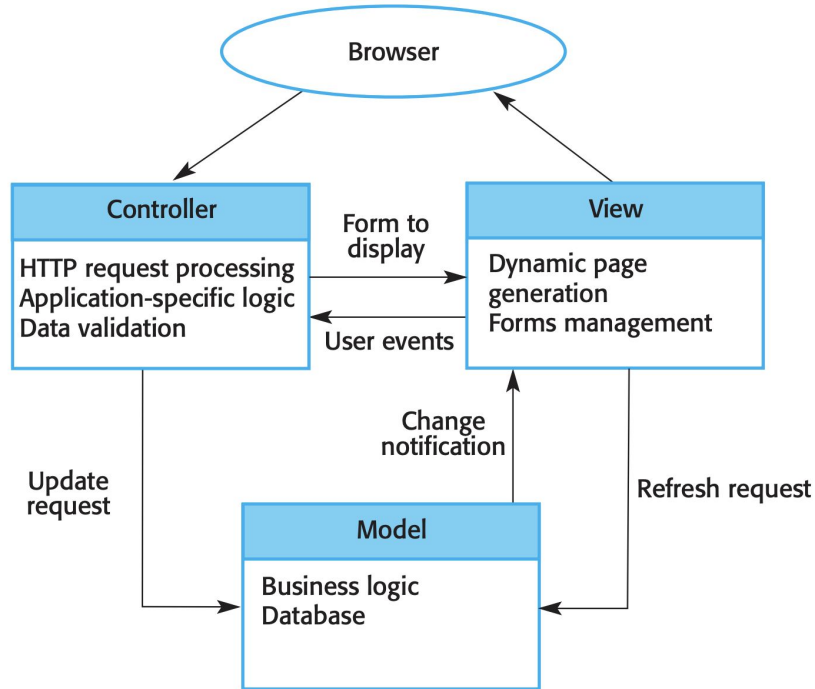
Design pattern for the architecture/organization of the server application

An application organized according to MVC consists of three main types of code:

3) **Controllers** (mediate the interaction in both directions)

- When a user interacts with a view, a specific controller action corresponding to that user activity is invoked.
- Each **controller** is associated with **one model**.
- The controller can ask the model to retrieve or modify information; based on the results of doing this, the controller decides what view will be presented next to the user, and supplies that view with any necessary information.

Model View Controller (MVC)



View _> HTML page

The HyperText Markup Language (**HTML**) is the **standard markup language for documents** designed to be **displayed in a web browser**.

Web browsers receive HTML documents from a web server and render them.

HTML describes the **structure of a web page** and comes with (some) cues for its appearance. It can be assisted by technologies such as CSS and scripting languages such as JavaScript.

Cascading Style Sheets (CSS) is a style sheet language for describing how to present a document written in a markup language like HTML. CSS is designed to enable the separation of presentation and content (e.g., layout, colors, and fonts)

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

The text between `<html>` and `</html>` describes the web page,

the text between `<body>` and `</body>` is the visible page content.

the tag `<div>` defines a division of the page used for easy styling.

Additionally, a `<meta>` element is used in between `<head>` and `</head>`, which can be used to define the webpage's metadata.

https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/HTML_basics